Algorithm Lab (CS2271)
Assignment 7

Department of Computer Science and Technology
Indian Institute of Engineering Science and Technology, Shibpur

Q1. Problem Statement

In this assignment, you will design and implement a smart navigation system for a city with 10 locations (vertices) such as schools, hospitals, and markets. The roads between these locations are represented as edges:

$$E = \{(u, v, w) \mid w_i > 0\}$$

where each edge has a positive travel cost (distance or time). Dynamically construct the directed graph and compute the shortest path from a selected source location using two different approaches: adjacency list representation and adjacency matrix representation.

Your task:

1. Graph Construction

Define the vertex set:

$$V = \{v_1, v_2, \ldots, v_{10}\}$$

Dynamically construct the graph by allowing the user to:

- Select any two vertices (u, v)
- Assign a positive weight $w_i > 0$

Represent the same graph in two formats:

- Adjacency List Representation
- Adjacency Matrix Representation

2. Scenario 1: Adjacency List + Min-Heap

- Implement Dijkstra's Algorithm using adjacency list
- Use a min-heap (priority queue) to select the next vertex with minimum distance
- Compute the shortest path from a chosen source vertex to all other vertices

3. Scenario 2: Adjacency Matrix + Min-Heap

- Implement Dijkstra's Algorithm using adjacency matrix
- Use a min-heap for selecting the next vertex
- Compute the shortest path from the same source vertex

4. Performance Evaluation

Execute both implementations on the same graph input and measure:

- Execution time t

Perform analysis under:

- Sparse Graph (few edges)
- Dense Graph (many edges)

Q2. Problem Statement

In this assignment, you will explore an important limitation of Dijkstra's Algorithm by designing a graph with $V = 10$ vertices where the user dynamically constructs edges

$$E = \{(u, v, w)\},$$

including the possibility of **negative edge weights** and even a **negative weight cycle**.

While Dijkstra's Algorithm assumes

$$w_i > 0,$$

you will intentionally violate this condition by allowing

$$w_i < 0$$

and analyze how the algorithm fails to compute correct shortest paths from a source vertex s.

Dijkstra relies on the greedy assumption that once a node u is selected with minimum distance, its shortest path is finalized. However, in the presence of negative edges, this assumption breaks:

$$\exists (u, v) \in E \text{ such that } w(u, v) < 0 \Rightarrow dist[v] < dist[u] + w(u, v)$$

even after node u has already been processed.

Negative Cycle Condition

If a negative cycle exists such that:

$$\sum_{(u,v)\in C} w(u, v) < 0,$$

then the shortest path becomes undefined because distances can decrease indefinitely:

$$dist[v] \to -\infty$$

Your task:

- Dynamically create a graph with 10 vertices
- Add edges between vertices, including negative weights
- Intentionally create a cycle with total negative weight
- Apply Dijkstra's Algorithm from a source vertex
- Observe incorrect results or failure to detect the negative cycle

Q3. Problem Statement

You are given a set of n points:

$$S = \{p_1, p_2, \ldots, p_n\}, \quad \text{where } p_i = (x_i, y_i)$$

Your goal is to find the pair of points (p_i, p_j) that have the **smallest distance** among all pairs. The assignment considers two cases:

Case 1 – One-Dimensional Points (1D)

Each point is represented as:

$$p_i = (x_i)$$

The distance between any two points is calculated as:

$$d = |x_i - x_j|$$

You are required to find the pair of points with the minimum distance in 1D.

Case 2 – Two-Dimensional Points (2D)

Each point is represented as:

$$p_i = (x_i, y_i)$$

The distance between any two points is calculated using the Euclidean formula:

$$d = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

You are required to find the pair of points with the minimum distance in 2D.

Your tasks

1. Implement a method to find the closest pair of points for 1D points.
2. Implement a method to find the closest pair of points for 2D points.
3. Compare the time complexity of the 1D and 2D implementations

Q4. Problem Statement

In this assignment, you are given a **weighted directed graph**

$$G = (V, E)$$

where V is the set of vertices and E is the set of edges. Each edge $(u, v) \in E$ has a weight $w(u, v)$, which can be either positive or negative. Your task is to explore the behavior of the **Bellman-Ford Algorithm** when the graph contains a **negative weight cycle**.

The Bellman-Ford algorithm is used to find the **shortest path** from a source vertex s to all other vertices. The distance from s to any vertex v is calculated as:

$$d(v) = \min_{(u,v)\in E} \big(d(u) + w(u, v)\big)$$

for all edges $(u, v) \in E$.

If the graph contains a **negative weight cycle** (i.e., a cycle whose total weight is negative):

$$\sum_{(u,v)\in \text{cycle}} w(u, v) < 0$$

the Bellman-Ford algorithm can **detect the presence of the cycle** after $|V| - 1$ relaxations. This is done by checking if any edge can still reduce the distance:

If $d(u) + w(u, v) < d(v)$ after $|V| - 1$ iterations, a negative weight cycle exists.

However, when a negative weight cycle is present, the algorithm **cannot produce correct shortest path values**, because the distance can decrease indefinitely as the cycle is traversed repeatedly.

Your tasks

1. Implement the **Bellman-Ford algorithm** for a weighted directed graph.
2. Detect whether a **negative weight cycle** exists using the final relaxation check.